Name : Vaishnavi Eknath Avhad
Class : D20C
Roll No. : 2

Practical 5

Aim : Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory

1. Primitive Data Types, Variables and Functions

Primitive data types are the basic building blocks of Solidity smart contracts. The commonly used types are :

- uint / int : Used to store unsigned and signed integers of different sizes, such as uint256 and int128.
- bool : Represents a logical value, either true or false.
- address : Stores a 20-byte Ethereum account or contract address.
- bytes / string : Used to store binary data and textual data respectively.

Types of Variables

Solidity variables are mainly classified into :

- State variables : Stored permanently on the blockchain.
- Local variables : Temporary variables that exist only during function execution.
- Global variables : Predefined variables provided by Solidity, such as msg.sender, msg.value, and block.timestamp.

Functions

Functions are used to execute the logic of a smart contract. Two important function types are :

- pure : Cannot read or modify blockchain state. They operate only on the provided inputs and internal calculations.
- view : Can read blockchain state but cannot modify it.

Using the appropriate function type helps maintain contract integrity and can reduce unnecessary gas consumption.

2. Inputs and Outputs to Functions

Solidity functions can accept input parameters and return output values.

- Inputs : Allow users or other contracts to provide data to a function.
- Outputs : Allow a function to return the result of a calculation or operation.

For example, a function may accept an Ether amount as input and return whether a transfer was successful.
Solidity also supports named return variables, which make the code easier to understand, maintain, and debug.

3. Function Visibility, Modifiers and Constructors

Function Visibility

Visibility determines who can access a function.

- public : Can be called from inside and outside the contract.
- private : Can be accessed only within the contract where it is defined.
- internal : Can be accessed within the contract and by derived (child) contracts.
- external : Designed to be called from outside the contract, such as by external accounts or other contracts.

Modifiers

Modifiers are reusable blocks of code that control or modify the behavior of functions.
They are commonly used for access control. For example, an onlyOwner modifier can restrict a function so that only the contract owner can execute it.

Constructors

A constructor is a special function that runs only once, when the contract is deployed.
It is mainly used to initialize important contract values, such as assigning the deploying account as the contract owner.

4. Control Flow : if-else and Loops

Solidity provides control-flow statements similar to those found in other programming languages.

- if-else
    - The if-else statement is used for decision-making.
    - Example : A contract can check whether a user's balance is sufficient before allowing a transfer.
- Loops
    - Solidity supports :
    - for
    - while
    - do-while
    - Loops are used when the same operation needs to be performed repeatedly, such as iterating through an array of users.
    - However, loops should be used carefully because every iteration consumes gas. A large number of iterations can make a transaction expensive or even cause it to run out of gas.

5. Data Structures : Arrays, Mappings, Structs and Enums

- Arrays
    - Arrays store an ordered collection of elements. They can be either fixed-size or dynamic.
    - Example : An array can store the addresses of all registered users.
- Mappings
    - Mappings store data as key-value pairs and are useful for quick data lookups.
    - Example :
    - mapping(address => uint) balances;
    - This can be used to associate each user's address with their balance.
    - Unlike arrays, mappings do not support direct iteration.
- Structs
    - Structs allow multiple related values to be grouped into a single custom data type.
    - Example :
    - struct Player {
    - string name;
    - uint score;
    - }
- Enums
    - Enums define a fixed set of predefined values. They improve code readability when a variable can have only a limited number of states.
    - Example :
    - enum Status {
    - Pending,
    - Active,
    - Closed
    - }

6. Data Locations

Solidity provides three main data locations :

- Storage
    - Stores data permanently on the blockchain.
    - Mainly used for state variables.
    - Data remains available between function calls.
- Memory
    - Stores temporary data.

- Exists only during function execution.
- Commonly used for temporary local variables and function parameters.

- Calldata

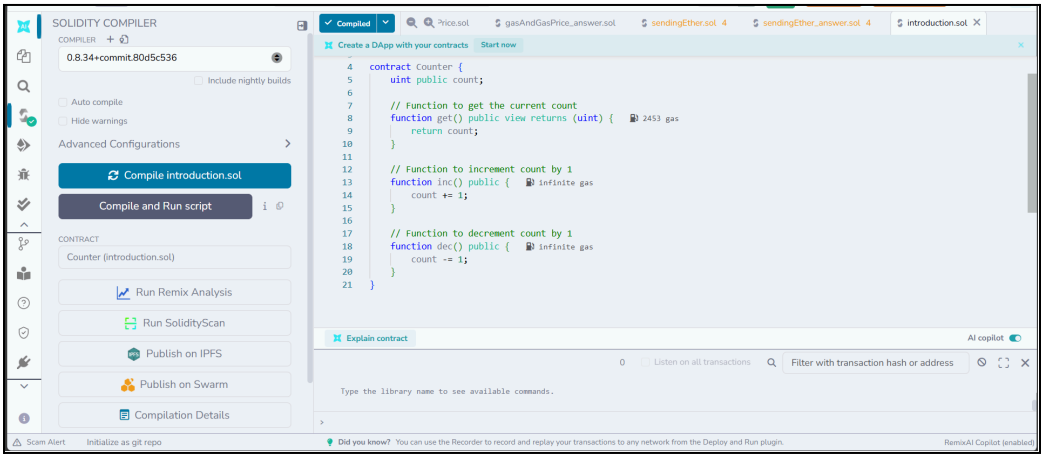
  - A temporary and read-only data location.
  - Mainly used for parameters of external functions.
  - Generally more gas-efficient than memory for external function inputs.
  - Understanding data locations is important because the choice of location affects gas consumption, performance, and how data is handled.

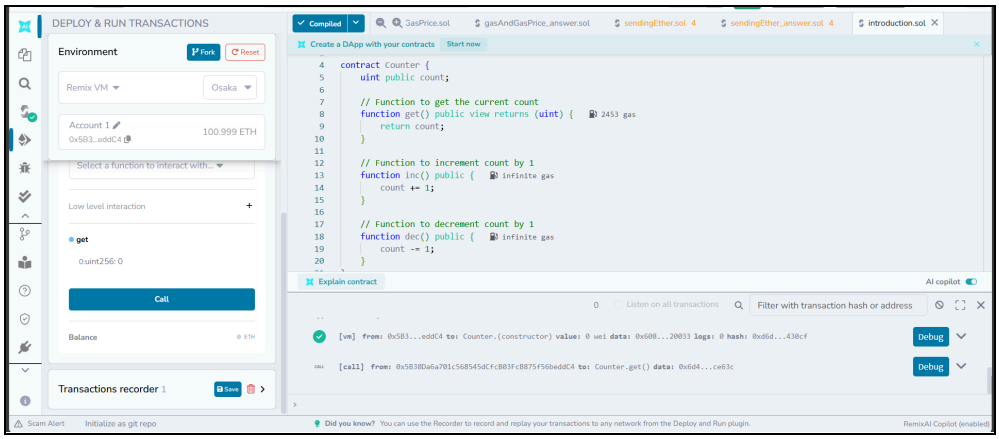
7. Transactions: Ether, Wei, Gas and Gas Price

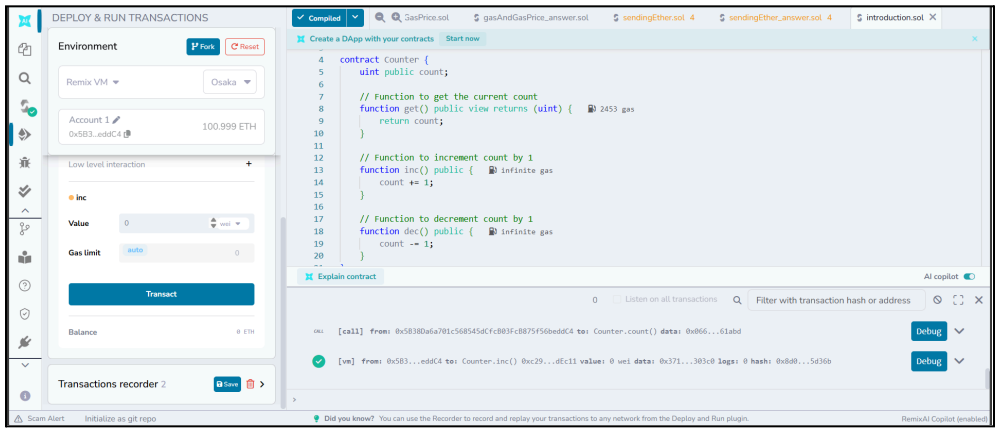
- Ether and Wei
  - Ether (ETH) is the primary currency of the Ethereum network.
  - Ether is divided into Wei, which is the smallest unit of Ether.
  - 1 Ether = 10^{18} Wei
  - Using Wei allows Ethereum to represent very small values accurately during financial transactions.
- Gas and Gas Price
  - Every transaction or smart contract operation requires gas, which represents the computational resources needed to execute it.
  - The gas price determines the amount of Ether paid for each unit of gas.
  - Therefore, transactions that require more computation generally consume more gas.
- Sending Transactions
  - Transactions can be used to :
  - Transfer Ether between accounts.
  - Interact with smart contracts.
  - Execute contract functions that modify blockchain state.
- Solidity provides methods such as transfer(), send(), and call() for transferring Ether.
  - Among these, call() provides greater flexibility and is commonly preferred for low-level Ether transfers.
  - Since transactions consume gas, writing efficient Solidity code is important for reducing execution costs.

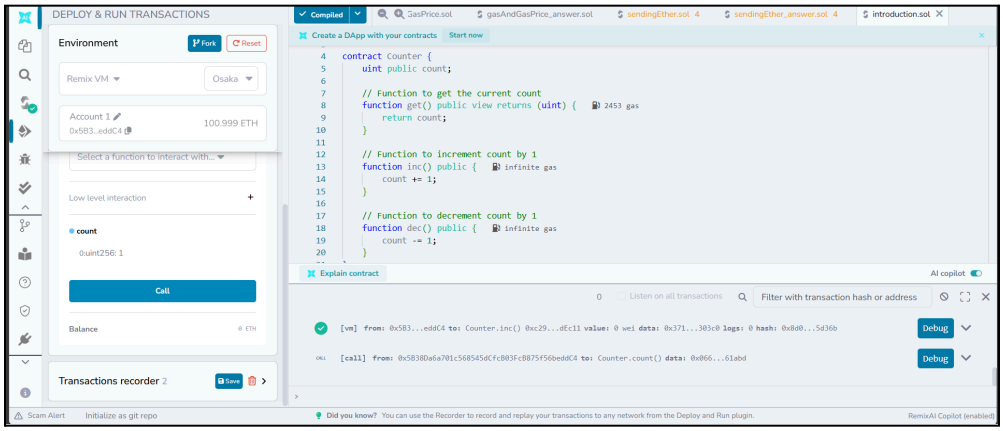
Output

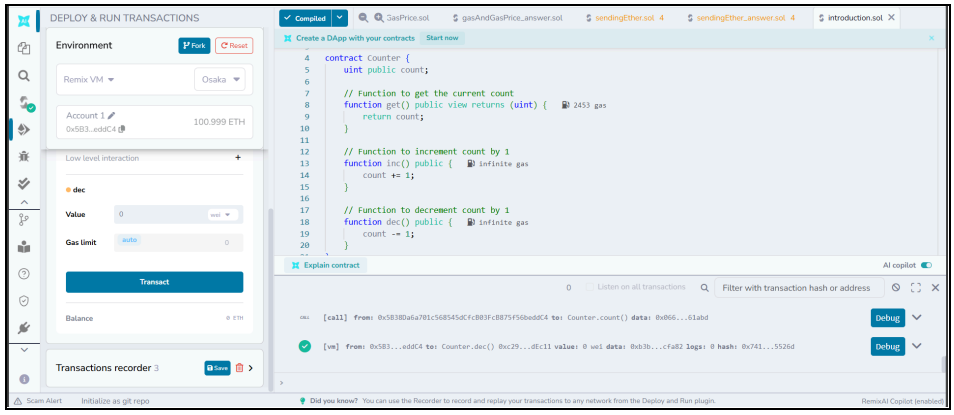
Tutorial No. 1 :

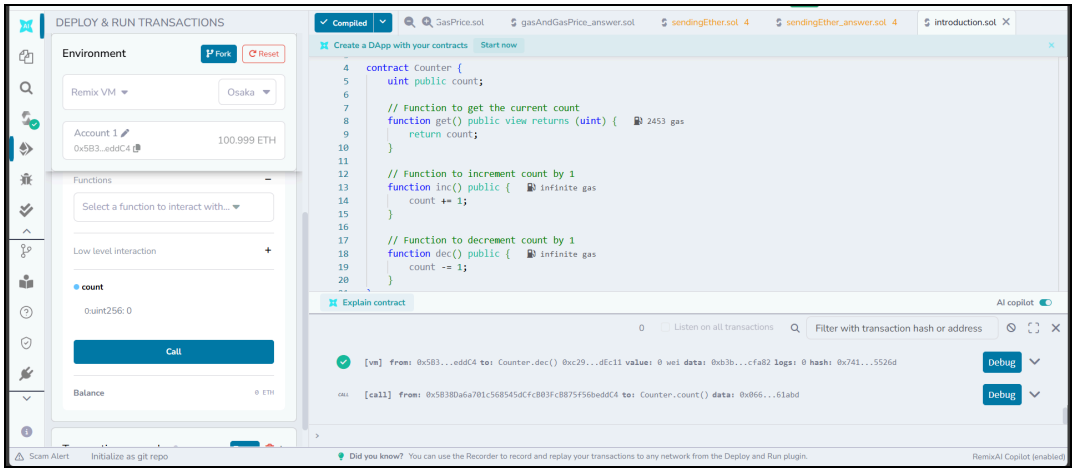



DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
100.999 ETH
Select a function to interact with...
Low level interaction
get
0:uint256: 0
Call
Balance
Transactions recorder 1
Save
Compiled
GasPrice.sol
gasAndGasPrice_answer.sol
sendingEther.sol 4
sendingEther_answer.sol 4
introduction.sol
Create a DApp with your contracts
Start now
contract Counter {
uint public count;
// Function to get the current count
function get() public view returns (uint) {
2453 gas
return count;
}
// Function to increment count by 1
function inc() public {
infinite gas
count += 1;
}
// Function to decrement count by 1
function dec() public {
infinite gas
count -= 1;
}
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or address
[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xd6d...430cf
Debug
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4...ce63c
Debug
Scam Alert
Initialize as git repo
Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.
RemixAI Copilot (enabled)

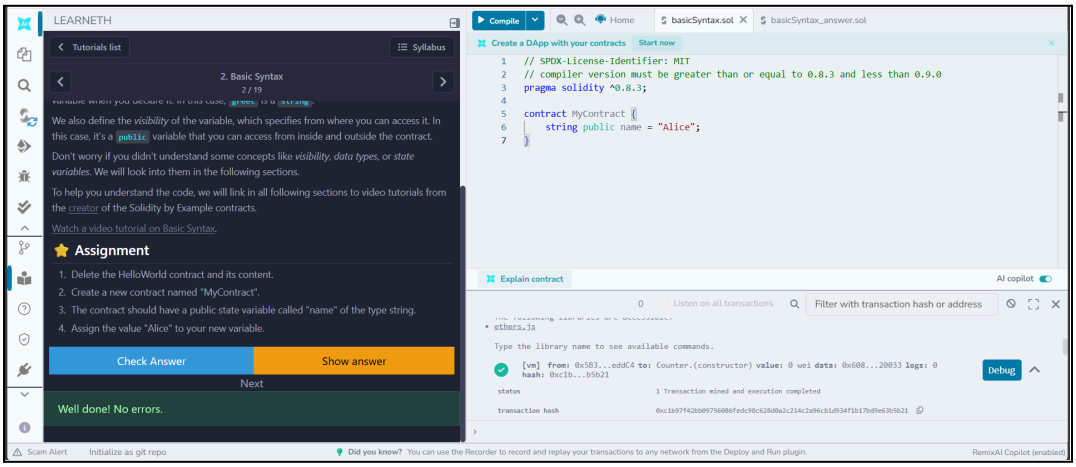

DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
100.999 ETH
Low level interaction
inc
Value
0
wei
Gas limit
auto
0
Transact
Balance
Transactions recorder 2
Save
Compiled
GasPrice.sol
gasAndGasPrice_answer.sol
sendingEther.sol 4
sendingEther_answer.sol 4
introduction.sol
Create a DApp with your contracts
Start now
contract Counter {
uint public count;
// Function to get the current count
function get() public view returns (uint) {
2453 gas
return count;
}
// Function to increment count by 1
function inc() public {
infinite gas
count += 1;
}
// Function to decrement count by 1
function dec() public {
infinite gas
count -= 1;
}
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or address
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.count() data: 0x066...61abd
Debug
[vm] from: 0x5B3...eddC4 to: Counter.inc() 0xc29...dEc11 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x8d0...5d36b
Debug
Scam Alert
Initialize as git repo
Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.
RemixAI Copilot (enabled)


DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
100.999 ETH
Select a function to interact with...
Low level interaction
count
0:uint256: 1
Call
Balance
Transactions recorder 2
Save
Compiled
GasPrice.sol
gasAndGasPrice_answer.sol
sendingEther.sol 4
sendingEther_answer.sol 4
introduction.sol
Create a DApp with your contracts
Start now
contract Counter {
uint public count;
// Function to get the current count
function get() public view returns (uint) {
2453 gas
return count;
}
// Function to increment count by 1
function inc() public {
infinite gas
count += 1;
}
// Function to decrement count by 1
function dec() public {
infinite gas
count -= 1;
}
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or address
[vm] from: 0x5B3...eddC4 to: Counter.inc() 0xc29...dEc11 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x8d0...5d36b
Debug
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.count() data: 0x066...61abd
Debug
Scam Alert
Initialize as git repo
Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.
RemixAI Copilot (enabled)

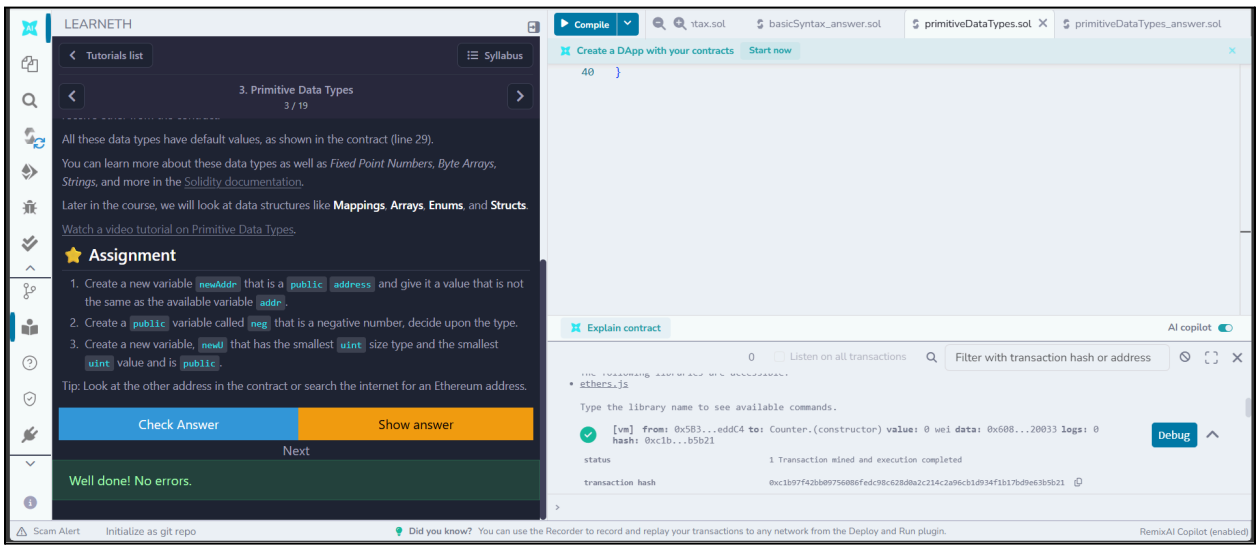

DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
100.999 ETH
Low level interaction
dec
Value
0
wei
Gas limit
auto
0
Transact
Balance
Transactions recorder 3
Save
Compiled
GasPrice.sol
gasAndGasPrice_answer.sol
sendingEther.sol 4
sendingEther_answer.sol 4
introduction.sol
Create a DApp with your contracts
Start now
contract Counter {
uint public count;
// Function to get the current count
function get() public view returns (uint) {
2453 gas
return count;
}
// Function to increment count by 1
function inc() public {
infinite gas
count += 1;
}
// Function to decrement count by 1
function dec() public {
infinite gas
count -= 1;
}
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or address
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.count() data: 0x066...61abd
Debug
[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xc29...dEc11 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0x741...5526d
Debug
Scam Alert
Initialize as git repo
Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.
RemixAI Copilot (enabled)



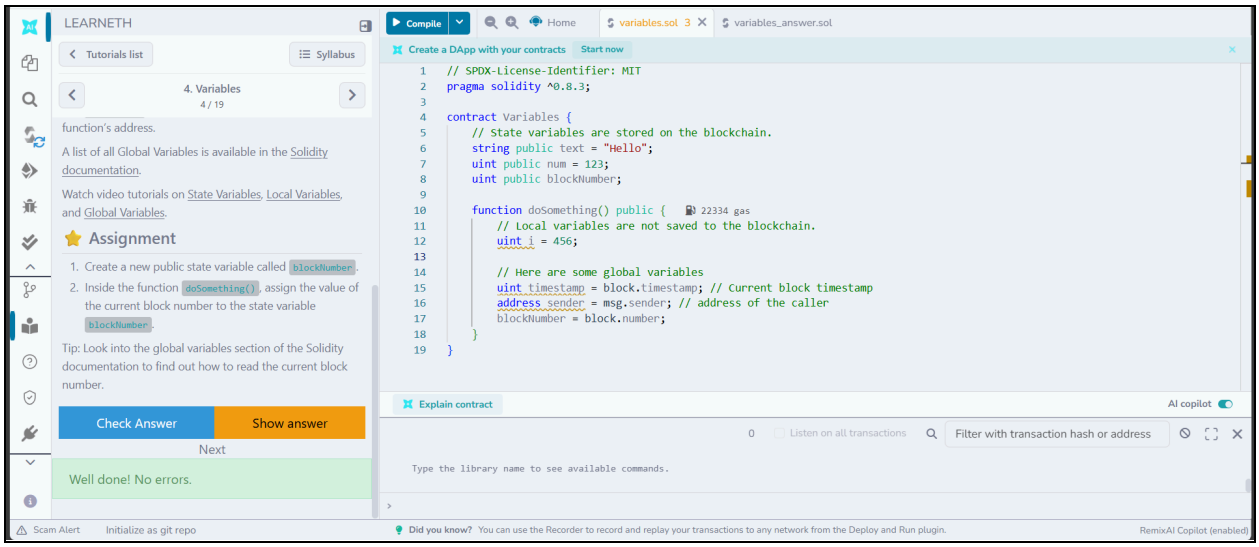
Tutorial No. 2 :



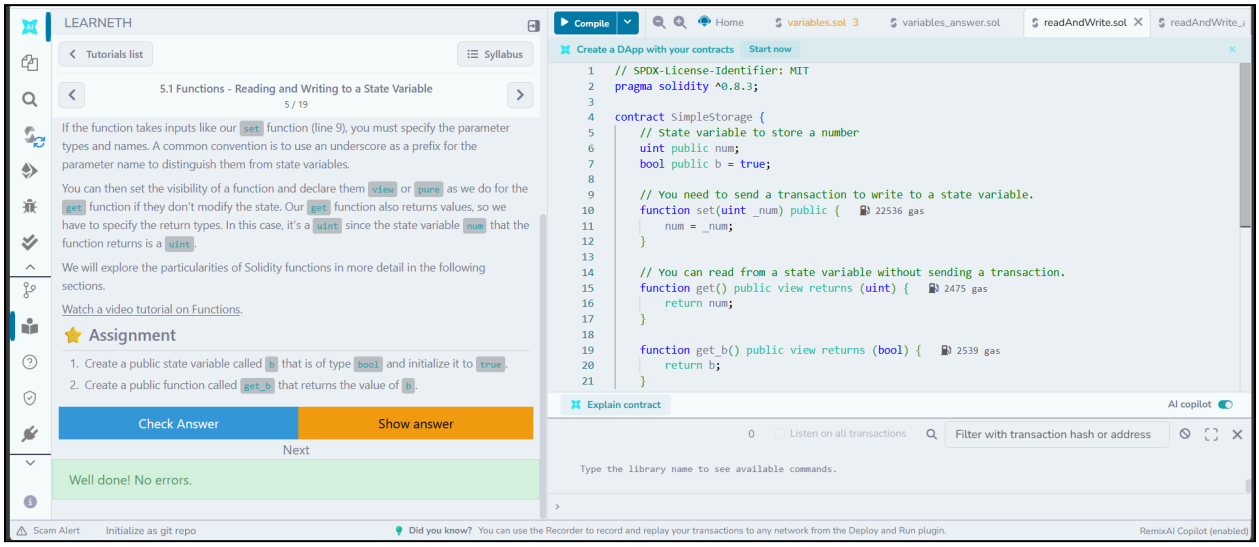
Tutorial No. 3 :



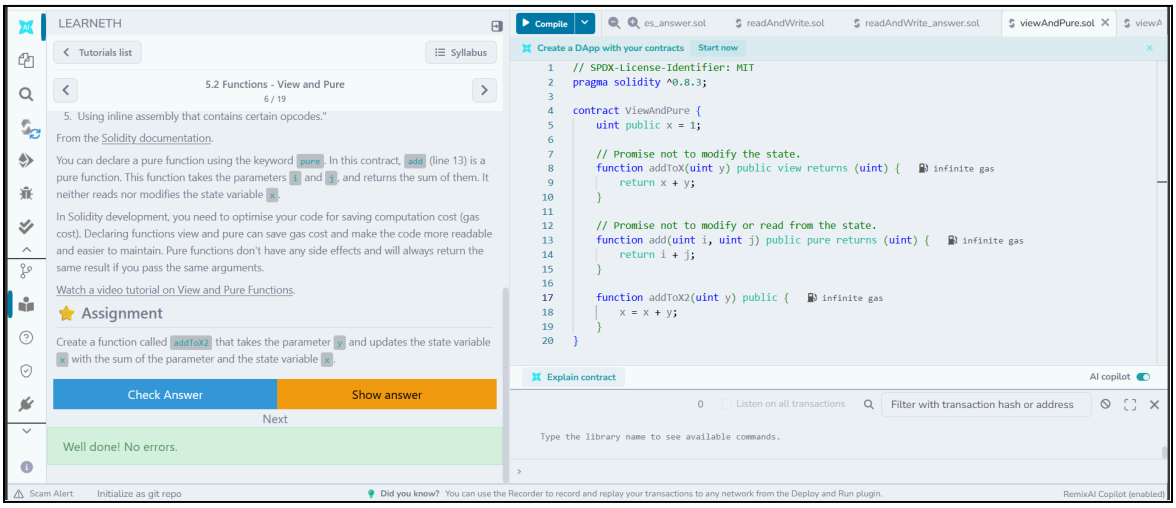
Tutorial No. 4 :



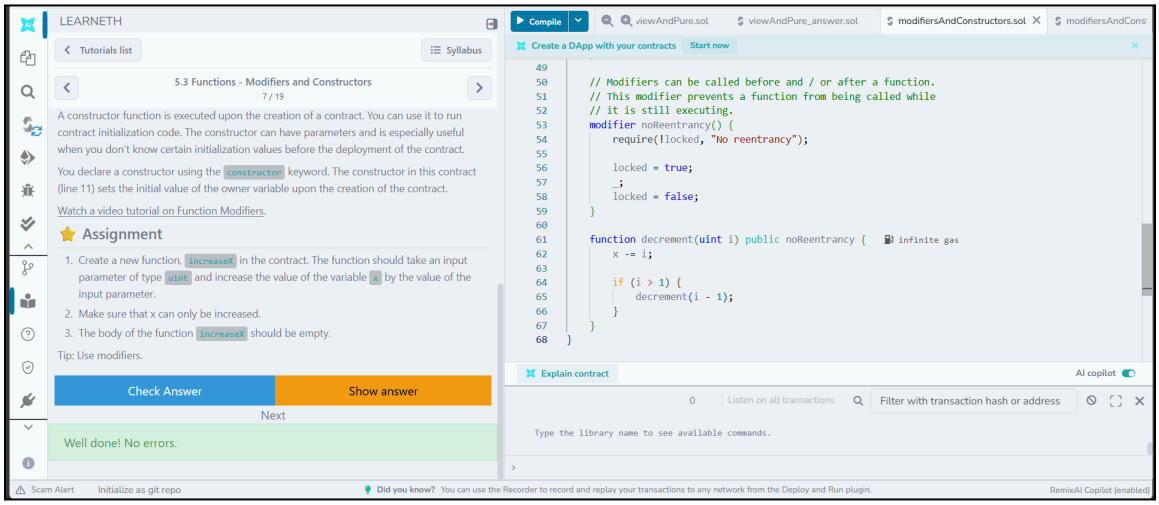
Tutorial No. 5 :



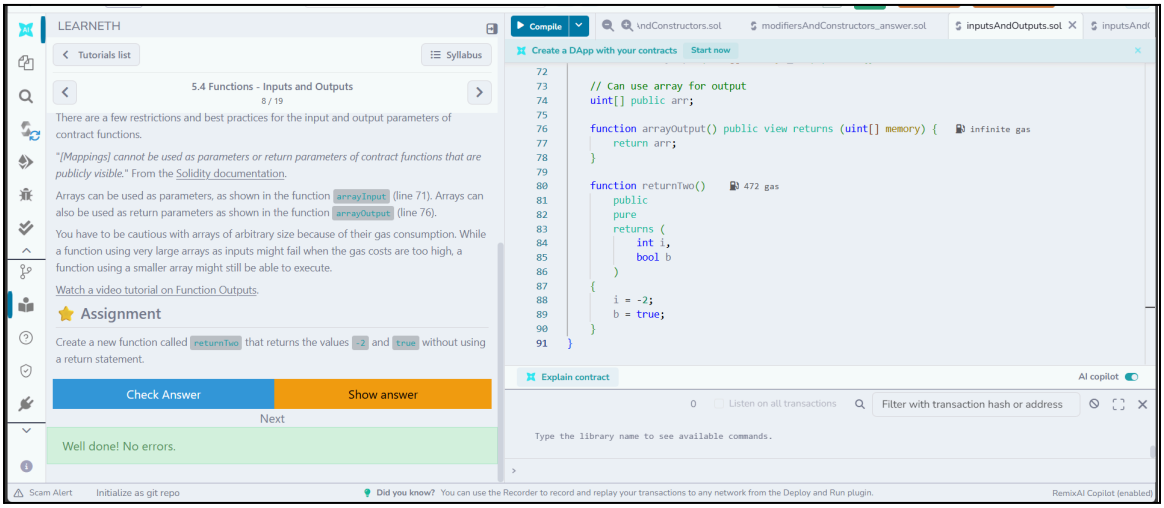
Tutorial No. 6 :



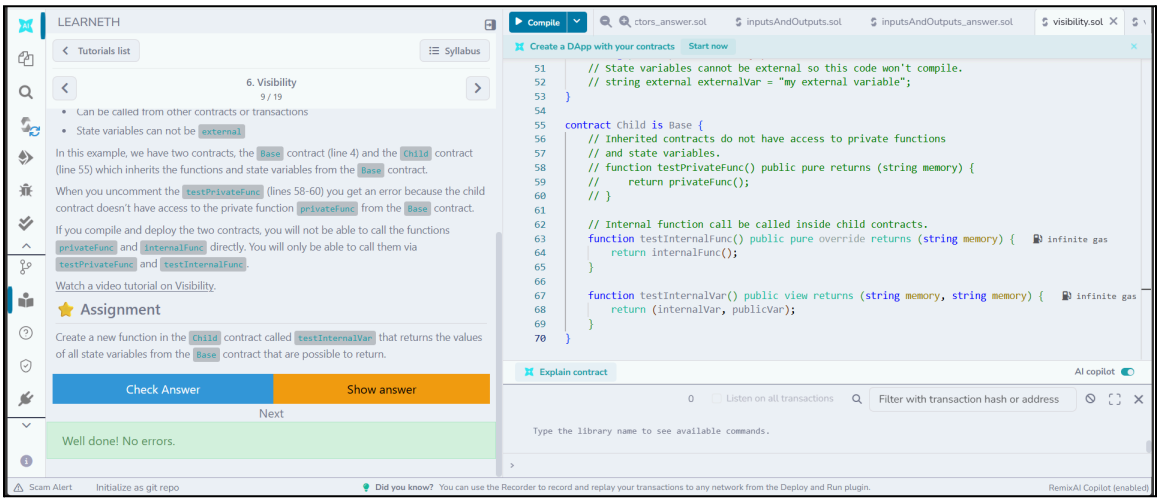
Tutorial No. 7 :



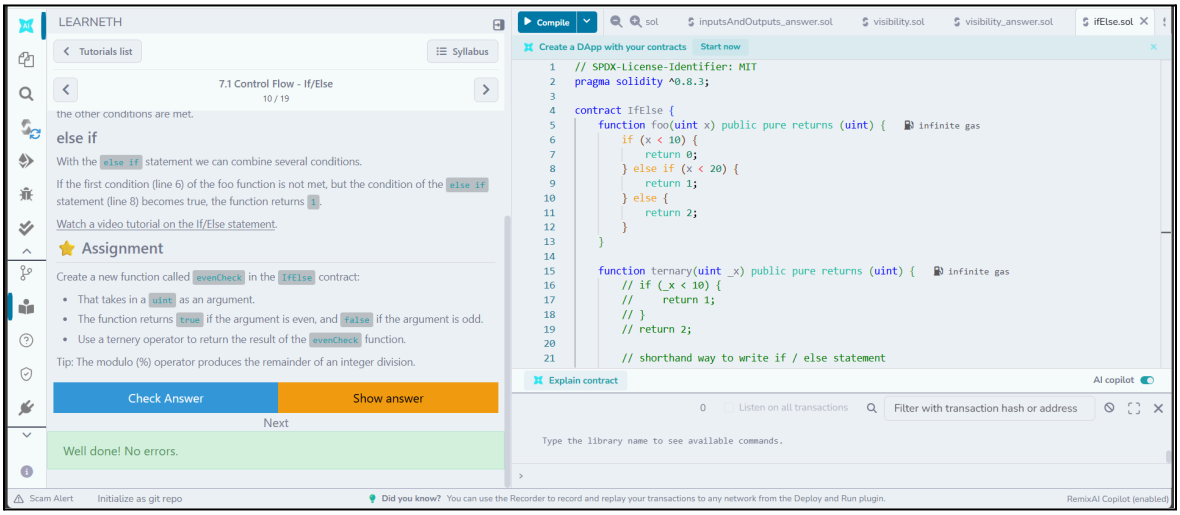
Tutorial No. 8 :



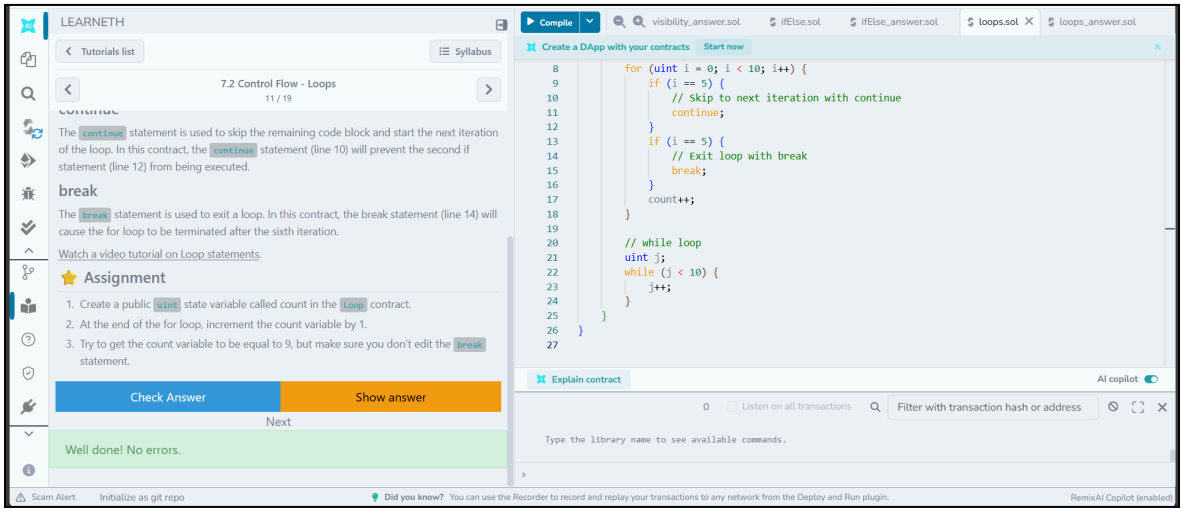
Tutorial No. 9 :



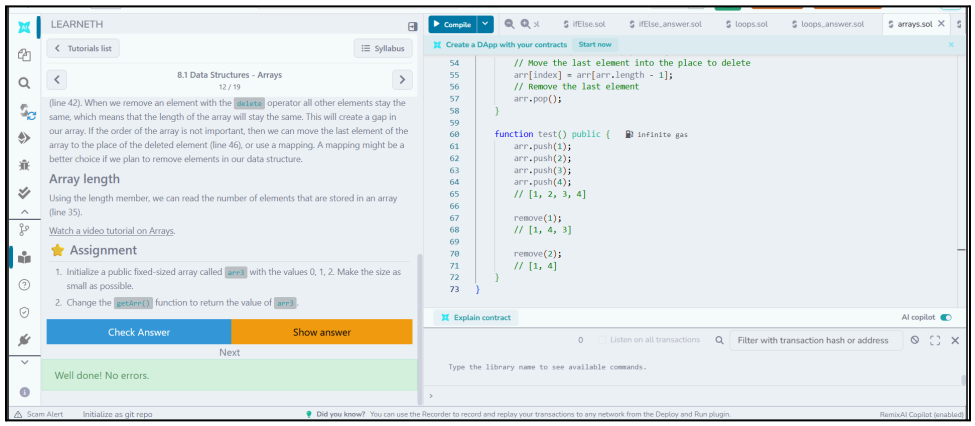
Tutorial No. 10 :



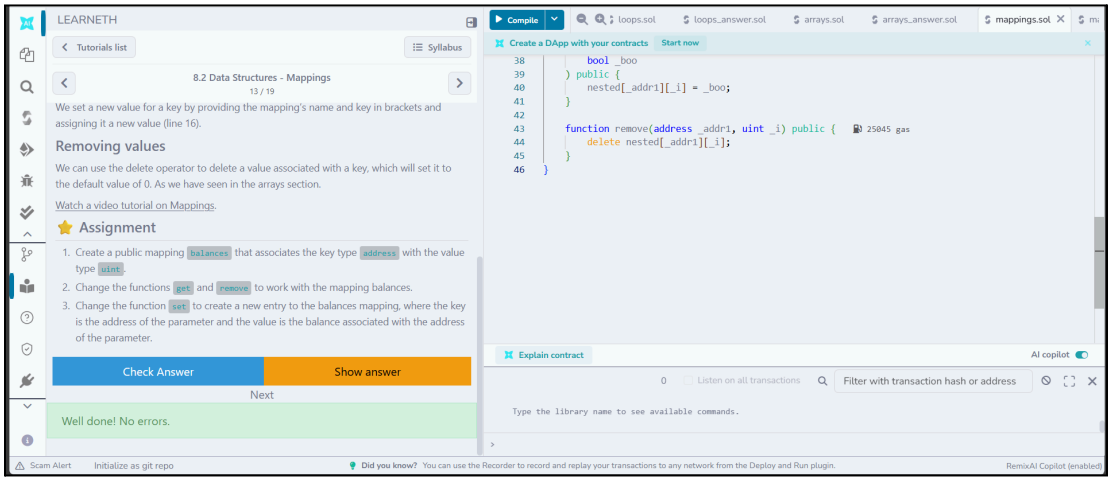
Tutorial No. 11 :



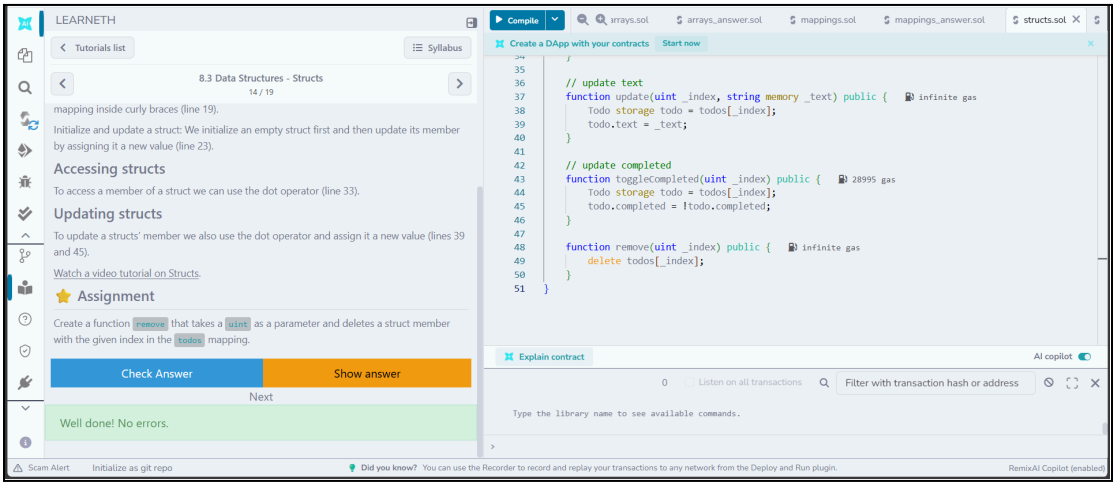
Tutorial No. 12 :



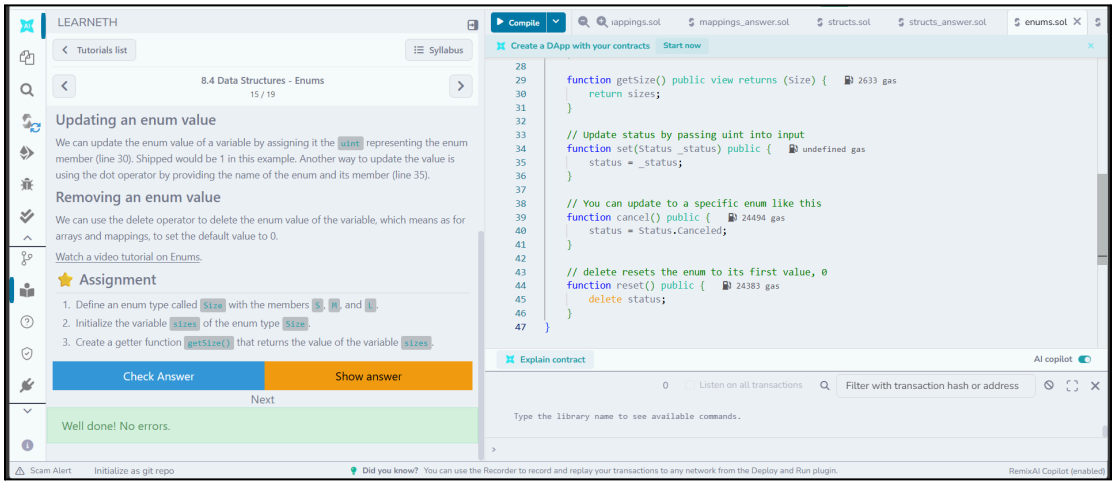
Tutorial No. 13 :



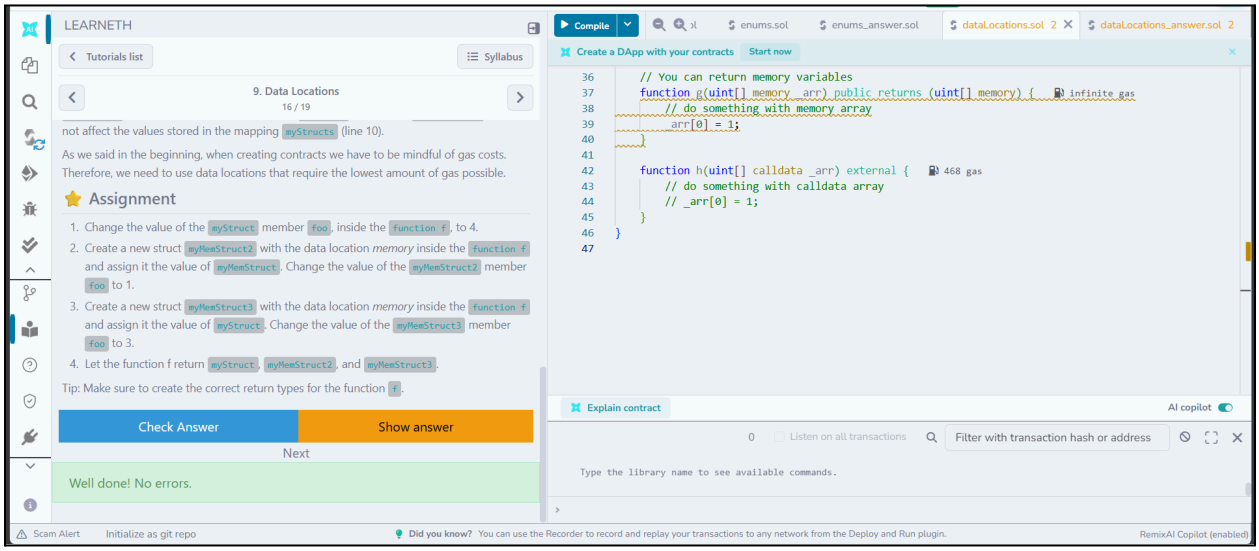
Tutorial No. 14 :



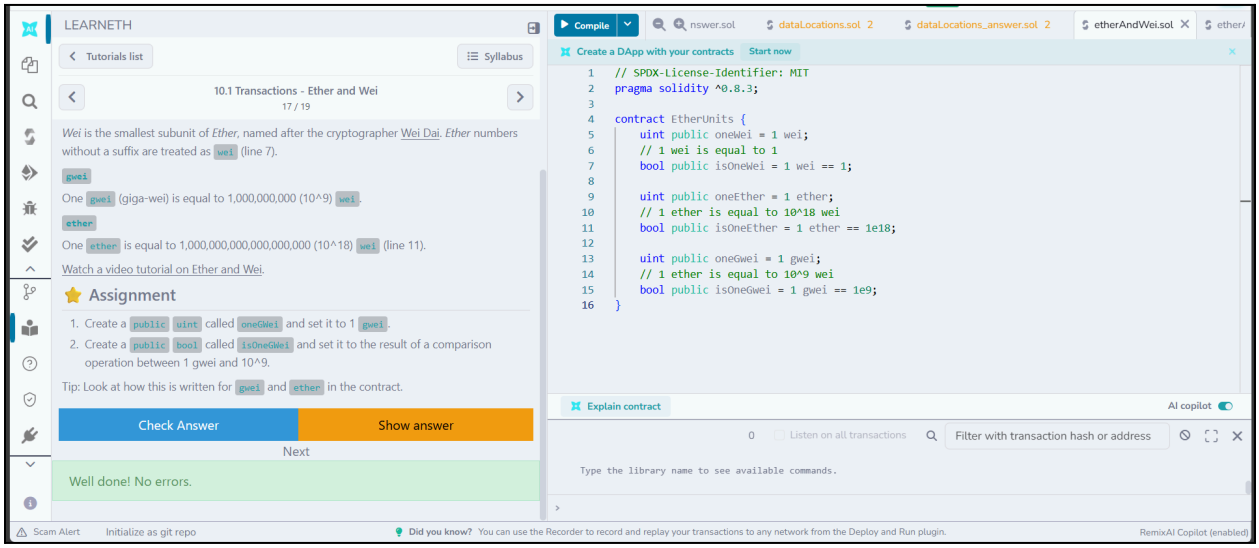
Tutorial No. 15 :



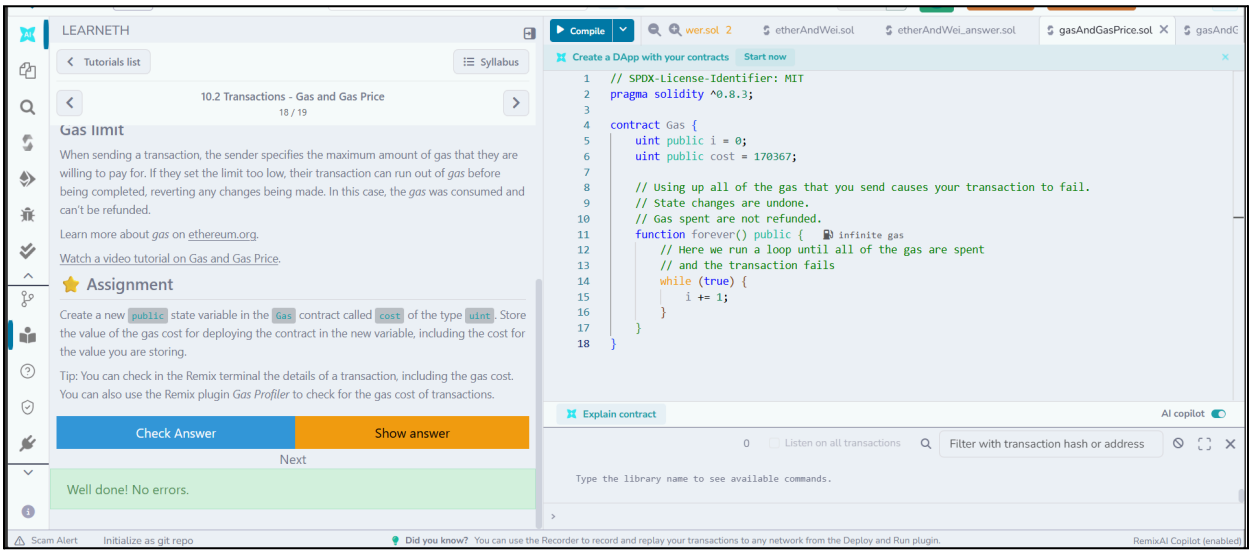
Tutorial No. 16 :



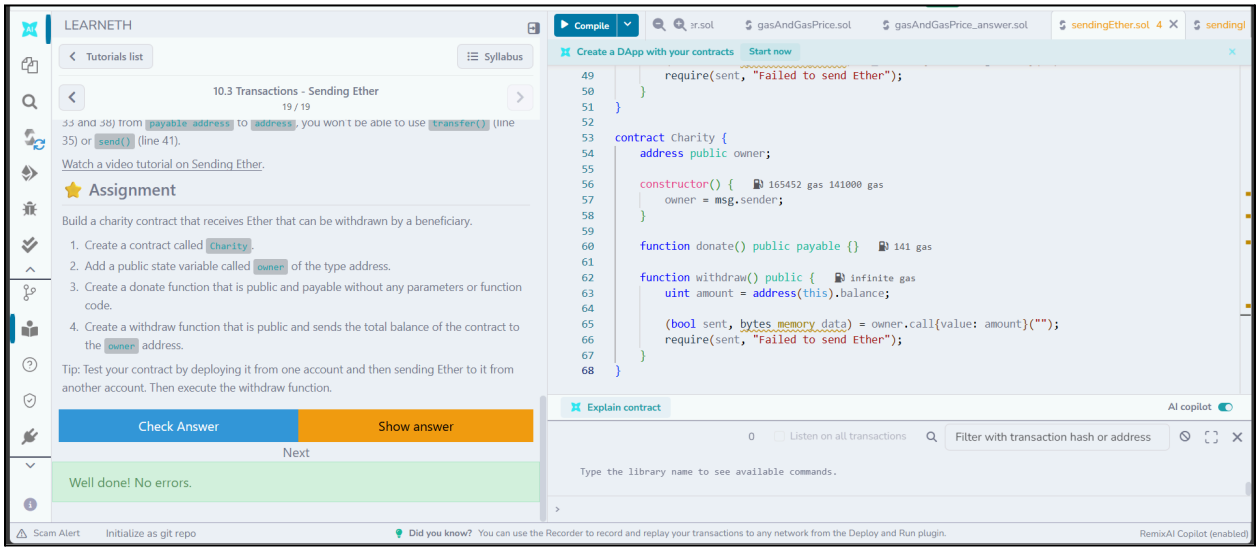
Tutorial No. 17 :



Tutorial No. 18 :



Tutorial No. 19 :



Conclusion

Thus, this practical provided hands-on experience with Solidity programming and the development of smart contracts. We learned about primitive data types, variables, functions, function visibility, modifiers, constructors, control-flow statements, data structures, data locations, and Ethereum transactions. The practical helped in understanding how Solidity is used to write secure, efficient, and reliable smart contracts on the Ethereum blockchain. Overall, the practical strengthened our basic knowledge and programming skills in blockchain and smart contract development.